

PI - Easypaisa REST APIs - Documentation

Document Number	OI-PO-20260430
Document status	PUBLISHED
Document owner	@Shayan Rasheed
Solutions Architect	@Maqsood Ali @Owais Khalid @Pawan Kumar
Priority	HIGH
Region	Pakistan
Operator	Easypaisa
Product	PAYIN

 [Easypaisa OPS API.postman_collection.json](#)  [API Integration Guide V1.3.pdf](#)

Table of Contents

1. [Overview](#)
2. [Credentials](#)
3. [Authentication](#)
 - 3.1 [Generate JWT Token](#)
 - 3.2 [Refresh JWT Token](#)
4. [Signature Generation](#)
 - 4.1 [Algorithm](#)
 - 4.2 [Step-by-Step Example](#)
 - 4.3 [Postman Pre-Request Script](#)
5. [API Endpoints](#)
 - 5.1 [Initiate MA Transaction](#)
 - 5.2 [IPN Flow](#)
 - 5.3 [Inquiry API](#)
 - 5.4 [Reversal API](#)
6. [Response Codes](#)

- 6.1 [Transaction Response Codes](#)
- 6.2 [Reversal-Specific Response Codes](#)
- 6.3 [Auth Response Codes](#)

1. Overview

This document describes the integration between **Simpaisa** (aggregator) and **Easypaisa Digital Bank**. It covers authentication, signature generation, transaction initiation, instant payment notification (IPN) handling, transaction inquiry, and reversal.

Supported Flows

Note: Currently the credentials are configured to Async flow. If Sync flow needed please ask the Product team to facilitate.

Flow	Description
Sync	Transaction response returned synchronously after customer approves the payment
Async	Transaction is created and immediately returns <code>040 - Payment Confirmation Pending</code> ; final status delivered via IPN

Base URLs

Environment	Base URL
Sandbox	<code>https://gateway-service-sandbox.easypaisa.com.pk</code>
Reversal URL for Sandbox	<code>https://gateway-service-sandbox.easypaisa.com.pk/reversal/</code>

2. Credentials

 **Security Notice:** The credentials below are sandbox credentials provided for integration and testing purposes only. Never commit these values to source control. Store them in environment variables or a secrets manager.

Sandbox Credentials

Parameter	Value	Notes
username	shayan.rasheed	Portal login username
password (plaintext)	SIMPAISA@123!	Used only to compute passwordMD5
passwordMD5	cba71e3ca4e0c74fd8fa0b8f2f78316f	MD5 of {"PASSWORD":"SIMPAISA@123!"} — sent as Password header
storeId	947842	Merchant store identifier
apiSecret	Provided separately by Easypaisa	Shared secret used in signature computation
externalSystemId	01	2-digit system identifier for reversals

How to Compute passwordMD5

The Password header must be the **MD5 hash of the JSON string** {"PASSWORD": "<your_plaintext_password>"}:

```
1 Input  : {"PASSWORD":"SIMPAISA@123!"}
2 Output : cba71e3ca4e0c74fd8fa0b8f2f78316f
3
```

In Python:

```
1 import hashlib, json
2 payload = json.dumps({"PASSWORD": "SIMPAISA@123!", separators=(',', ':')})
3 password_md5 = hashlib.md5(payload.encode()).hexdigest()
4 # → cba71e3ca4e0c74fd8fa0b8f2f78316f
5
```

3. Authentication

All API calls require a valid **JWT (JSON Web Token)** passed in the **Authorization** header as **Bearer <jwt>**.

- JWT validity: **20 minutes** from issuance. This can be configured to 24 hours.
- Expiry field: Unix timestamp (seconds)
- Best practice: Refresh the token before the expiry time of the JWT token

3.1 Generate JWT Token

Obtain a new JWT using your portal credentials.

Endpoint

```
1 POST /auth/generateJWT
2
```

Request Headers

Header	Value	Description
Username	{{username}}	Portal username
Password	{{passwordMD5}}	MD5 hash of {"PASSWORD": " <pass>"}

Note: No request body is required. Credentials are passed entirely via headers.

Sample cURL

```
1 curl --location --request POST \
2   'https://gateway-service-sandbox.easypaisa.com.pk/auth/generateJWT' \
3   --header 'Username: shayan.rasheed' \
4   --header 'Password: cba71e3ca4e0c74fd8fa0b8f2f78316f'
5
```

Success Response

```
1 {
2   "jwt": "eyJhbGciOiJIUzI1NiJ9...",
3   "responseCode": "000",
4   "expiry": "1769673951",
5   "responseMessage": "JWT is generated successfully"
6 }
```

Response Fields

Field	Type	Description
jwt	String	Bearer token to use in subsequent requests
responseCode	String	000 = success
expiry	String	Unix timestamp (seconds) when the token expires
responseMessage	String	Human-readable status

3.2 Refresh JWT Token(Not needed to integrate as you can use previous one to generate new token again)

Used to obtain a new JWT before or after the current one expires. Requires a valid (or recently expired) JWT as the `refreshToken`.

Endpoint

```
1 POST /auth/refreshJWT
2
```

Request Headers

Header	Value
Content-Type	application/json

Request Body

```
1 {
2   "request": {
3     "refreshToken": "<original_jwt>"
4   },
5   "signature": "<computed_signature>"
6 }
7
```

The `signature` must be computed using the same MD5 algorithm described in [Section 4](#), applied to the `request` object combined with the `apiSecret`.

Sample cURL

```
1 curl --location --request POST \  
2 'https://gateway-service-sandbox.easypaisa.com.pk/auth/refreshJWT' \  
3 --header 'Content-Type: application/json' \  
4 --data '{  
5     "request": {  
6         "refreshToken": "eyJhbGciOiJIUzI1NiJ9..."  
7     },  
8     "signature": "<computed_signature>"  
9 }'  
10
```

Success Response

```
1 {  
2     "jwt": "eyJhbGciOiJIUzI1NiJ9...",  
3     "refreshJwt": "eyJhbGciOiJIUzI1NiJ9...",  
4     "responseCode": "000",  
5     "expirationTime": "1769673951",  
6     "responseMessage": "JWT is generated successfully"  
7 }  
8
```

Response Fields

Field	Type	Description
<code>jwt</code>	String	New bearer token
<code>refreshJwt</code>	String	New refresh token for the next rotation
<code>expirationTime</code>	String	Unix timestamp (seconds) of new token expiry

4. Signature Generation

Every API request body (except JWT generation) must include a `signature` field. This is a cryptographic proof that the request payload has not been tampered with in transit.

4.1 Algorithm

The signature is an **MD5 hash** computed as follows:

```
1 signature = MD5( UPPERCASE( STRIP_SPACES( JSON.stringify(  
2   requestPayload + apiSecret ) ) ) )
```

Step-by-step:

1. Take the `request` payload object (the inner object, not the outer wrapper).
2. Append `"apisecret": "<your_api_secret>"` as an additional field.
3. Serialize the combined object to a JSON string.
4. Remove **all whitespace** from the string (spaces, tabs, newlines).
5. Convert the entire string to **UPPERCASE**.
6. Compute the **MD5 hash** of the uppercase string.
7. Place the resulting hex string in the `signature` field of the request body.

4.2 Step-by-Step Example

Input — Request Payload

```
1 {  
2   "msisdn": "923001234567",  
3   "orderId": "ORD-202501010001",  
4   "transactionAmount": "1000",  
5   "channel": "OPS",  
6   "storeId": "2113"  
7 }  
8
```

Step 1 — Append `apisecret`

```
1 {  
2   "msisdn": "923001234567",  
3   "orderId": "ORD-202501010001",  
4   "transactionAmount": "1000",  
5   "channel": "OPS",  
6   "storeId": "2113",  
7   "apisecret": "ABC123SECRET"  
8 }  
9
```

Step 2 — Stringify

```
1 {"msisdn":"923001234567","orderId":"ORD-  
2 202501010001","transactionAmount":"1000","channel":"OPS","storeId":"211  
3","apisecret":"ABC123SECRET"}  
2
```

Step 3 — Remove spaces *(already no spaces in the example above)*

Step 4 — Convert to UPPERCASE

```

1 {"MSISDN":"923001234567","ORDERID":"ORD-
  2 202501010001","TRANSACTIONAMOUNT":"1000","CHANNEL":"OPS","STOREID":"211
    3","APISECRET":"ABC123SECRET"}

```

Step 5 — Compute MD5

```

1 signature = MD5("<uppercased_string>")
2

```

Python Implementation

```

1 import json
2 import hashlib
3
4 def compute_signature(request_payload: dict, api_secret: str) -> str:
5     payload_with_secret = {**request_payload, "apisecret": api_secret}
6     stringified = json.dumps(payload_with_secret, separators=(',',
7     ':'))
8     no_spaces = stringified.replace(' ', '')
9     uppercased = no_spaces.upper()
10    return hashlib.md5(uppercased.encode()).hexdigest()
11
12 # Example
13 payload = {
14     "msisdn": "923001234567",
15     "orderId": "ORD-202501010001",
16     "transactionAmount": "1000",
17     "channel": "OPS",
18     "storeId": "2113"
19 }
20 signature = compute_signature(payload, "ABC123SECRET")
21 print(signature)
22

```

4.3 Postman Pre-Request Script

The following script is used in the Postman collection and automatically computes and injects the `signature` before each request is sent:

```

1 // Retrieve environment variables
2 const apiSecret = pm.environment.get('apiSecret');
3
4 // Parse the current request body
5 let bodyObj = JSON.parse(pm.request.body.raw);
6 const requestPayload = bodyObj.request;
7
8 // Append apiSecret to the payload
9 const payloadWithSecret = { ...requestPayload, apisecret: apiSecret };
10
11 // Stringify → Remove spaces → Uppercase → MD5
12 const stringified = JSON.stringify(payloadWithSecret);
13 const noSpaces = stringified.replace(/\s/g, "");
14 const uppercased = noSpaces.toUpperCase();
15 const signature = CryptoJS.MD5(uppercased).toString();
16
17 // Debug logs (visible in Postman Console)
18 console.log("1 - Stringified :", stringified);
19 console.log("2 - No spaces :", noSpaces);
20 console.log("3 - Uppercased :", uppercased);
21 console.log("4 - Signature :", signature);
22

```



```

23 // Save signature to environment and inject into request body
24 pm.environment.set('signature', signature);
25 bodyObj.signature = signature;
26 pm.request.body.update(JSON.stringify(bodyObj, null, 2));
27

```

! Important: The `apisecret` field is **never sent in the actual request body**. It is only used locally to compute the signature, then discarded. Only the resulting MD5 hash is transmitted in the `signature` field.

5. API Endpoints

All transaction endpoints share a common request envelope:

```

1 {
2   "request": { /* business payload */ },
3   "signature": "<computed_MD5>"
4 }
5

```

Common Request Headers

Header	Value	Description
Authorization	Bearer {{jwt}}	JWT obtained from Generate JWT
X-Forwarded-For	Client IP	Actual IP address of the calling client
Operation	ma / reversal	Constant per endpoint (see each section)
Content-Type	application/json	Fixed value

5.1 Initiate MA Transaction

Creates a Mobile Account (MA) payment transaction. Supports both **Sync** and **Async** flows depending on merchant configuration.

Flow	Behaviour	Response Code
Sync	Waits for customer approval; response	000 – Success

	returned after confirmation	
Async	Returns immediately after transaction creation; final status via IPN	040 – Payment confirmation pending

Endpoint

1	POST /transaction/initiate-ma
2	

Operation Header: ma

Request Parameters

Field	Description	M/O	Type	Max Length
orderId	Merchant-generated unique order ID	M	String	20
storeId	Store ID linked to partner account credentials	M	Long	6
transactionAmount	Total transaction amount	M	Numeric	10
channel	Transaction channel (e.g., APP)	M	String	10
msisdn	Customer mobile number — format: 03XXXXXXX	M	String	11

	XX (11 digits)			
emailAddress	Customer email — format: abc@xyz.com	M	String	250
signature	MD5 signature of request payload + apiSecret	M	String	—

Response Parameters

Field	Description	Type
orderId	Merchant's order ID	String
storeId	Merchant store ID	Long
transactionId	Easypaisa transaction ID (EWP ID)	String
transactionDateTime	Format: dd/MM/yyyy hh:mm a	String
responseCode	Platform-generated response code	String
responseDesc	Platform-generated response description	String

Sample — Sync Request

```

1  {
2    "request": {
3      "msisdn": "923413199000",
4      "orderId": "DemoR1T3",
5      "transactionAmount": 13,
6      "emailAddress": "customer@example.com",
7      "channel": "APP",
8      "storeId": "947842"
9    },
10   "signature": "{{computed_signature}}"

```

```
11 }
12
```

Sample — Sync Success Response

```
1 {
2   "orderId": "DemoR1T3",
3   "storeId": 43,
4   "transactionId": "253184",
5   "transactionDateTime": "11/08/2018 11:30 PM",
6   "responseCode": "000",
7   "responseDesc": "SUCCESS"
8 }
9
```

Sample — Async Success Response

```
1 {
2   "orderId": "DemoR1T3",
3   "storeId": 43,
4   "transactionId": "253184",
5   "transactionDateTime": "11/08/2018 11:30 PM",
6   "responseCode": "040",
7   "responseDesc": "payment confirmation pending"
8 }
9
```

Sample — Failure Response

```
1 {
2   "orderId": "ORD-20251031-001",
3   "storeId": "947842",
4   "transactionId": "0",
5   "transactionDateTime": "2025-10-31T14:30:00",
6   "responseCode": "013",
7   "responseDesc": "Merchant validation failed"
8 }
9
```

5.2 IPN Flow

The **Instant Payment Notification (IPN)** is how Easypaisa pushes the final transaction status to the merchant for async transactions.

Configuration

IPN settings are managed from the **Merchant Portal > Edit Merchant screen**. The following parameters are configurable:

Setting	Description
IPN Handle URL	Merchant's HTTPS endpoint to receive IPN notifications
Receive IPN Ind	Toggle to enable/disable IPN receipt

IPN Retry Enabled Ind	Whether to retry on failure
IPN Re-Attempts	Number of retry attempts (e.g., 3)
IPN Read Timeout	Timeout in ms for reading IPN response (e.g., 5000)
IPN Connection Timeout	Timeout in ms for establishing connection (e.g., 3000)
IPN Retry Wait Interval	Wait interval in seconds between retries (e.g., 5)
Enable Payment Link Ind	Toggle for payment link support

IPN Payload

The attributes included in the IPN notification are configurable from the **IPN Attributes screen** in the Merchant Portal. The system sends the configured attributes as a JSON object:

```

1  {
2    "transaction_id": "",
3    "transaction_status": "",
4    "msisdn": "",
5    "payment_method": "",
6    "store_name": "",
7    "transaction_amount": "",
8    "order_id": "",
9    "order_datetime": ""
10 }
11

```

IPN Response Codes (Merchant → Easypaisa)

Upon receiving the IPN, the merchant system must respond with one of:

Code	Meaning
00	Transaction accepted
01	Transaction rejected / dropped
02	Transaction accepted via admin portal

Retry Mechanism

If the IPN endpoint is unreachable, returns HTTP 500 , or encounters a connection error, the system retries the IPN according to the retry configuration defined in the Merchant Portal.

Flow Logic Based on IPN Response

Merchant IPN Response	Action
00 — Accepted	No further action; transaction marked complete
01 — Rejected + Auto-Reversal enabled	Transaction is reversed automatically
01 — Rejected + Auto-Reversal disabled	Transaction marked for reversal; settled manually via admin portal
Admin portal update	IPN received value set to 02 in transaction table

5.3 Inquiry API

Retrieves the status of one or more transactions after completion. At least one of `orderId`, `opsId`, or `transactionId` must be provided. `transactionDate` is optional but must be paired with at least one other identifier.

Endpoints

1	POST /transaction/inquiry	← Returns latest single transaction
2	POST /inquiry/list	← Returns all transactions for an
3	orderId	

Operation Header: ma

Request Parameters

Field	Description	M/O	Type	Max Length
storeId	Merchant store ID	M	String	6
orderId	Order ID to search	O*	String	20
transactionDate	Date of transaction — format:	O	Date	10

	DD/MM/YYYY Y			
opsId	OPS Transaction ID	O*	String	10
transactionId	EWP Transaction ID	O*	String	20

*At least one of `orderId`, `opsId`, or `transactionId` must be present.

Sample Request

```

1 {
2   "request": {
3     "storeId": "947842",
4     "transactionDate": "22/04/2026",
5     "opsId": "10912390"
6   },
7   "signature": "{{computed_signature}}"
8 }
9

```

Sample Response — Single Inquiry

```

1 {
2   "responseCode": "000",
3   "responseDesc": "Success",
4   "transaction": {
5     "orderId": "OnlineSettle",
6     "storeId": "947842",
7     "transactionId": null,
8     "OPSID": "144",
9     "transactionStatus": "FAILED",
10    "transactionAmount": 3100000000,
11    "transactionDateTime": "30/12/2025 04:58:33 PM",
12    "msisdn": "923413199000",
13    "reversal_status": null,
14    "settlementStatus": null,
15    "reversalAmount": null,
16    "errorCode": "FAILED",
17    "errorReason": "Insufficient balance"
18  }
19 }
20

```

Sample Response — List Inquiry

```

1 {
2   "responseCode": "000",
3   "responseDesc": "Success",
4   "transactions": [
5     {
6       "orderId": "OnlineSettle",

```

```

7      "storeId": "947842",
8      "transactionId": null,
9      "OPSID": "144",
10     "transactionStatus": "FAILED",
11     "transactionAmount": 3100000000,
12     "transactionDateTime": "30/12/2025 04:58:33 PM",
13     "msisdn": "923413199000",
14     "reversal_status": null,
15     "settlementStatus": null,
16     "reversalAmount": null,
17     "errorCode": "FAILED",
18     "errorReason": "Insufficient balance"
19   }
20 ]
21 }
22

```

Response Parameters

Field	Description	Type
responseCode	Response code	String
responseDesc	Response description	String
transaction / transactions	Single object or array	Object / Array
orderId	Merchant's order ID	String
storeId	Store ID	String
transactionId	EWP transaction ID	String
OPSID	OPS transaction ID	String
transactionStatus	Status of the transaction	String
transactionAmount	Transaction amount	String
transactionDateTime	Timestamp of transaction	String
msisdn	Mobile account used	String
reversal_status	Reversal status (if applicable)	String
reversalAmount	Amount reversed (if applicable)	String

settlementStatus	Settlement status	String
errorCode	Error code for failed transactions	String
errorReason	Error reason for failed transactions	String

5.4 Reversal API

Enables merchants to initiate a partial or full reversal of a completed transaction.

⚠ Note: The Reversal API uses a separate base URL: `http://gateway-service-opssit.telenorbank.pk`

Endpoint

```
1 POST /reversal/reverse-transaction
2
```

Operation Header: reversal

Request Parameters

Field	Description	M/O	Type	Max Length
externalSystemId	2-digit external system identifier	M	String	2
storeId	Merchant store ID	M	String	6
reversalAmount	Amount to reverse	M	String	10
orderId	Order ID to reverse	M	String	20
orderDate	Date of original transaction	M	Date	10

	— format: DD/MM/YYYY Y			
cnic	Customer CNIC (13 digits, no dashes)	O	String	13
transactionId	EWP transaction ID of the original transaction	O	String	20

Sample Request

```

1  {
2    "request": {
3      "externalSystemId": "01",
4      "storeId": "947842",
5      "reversalAmount": "25",
6      "orderId": "OnlineService1",
7      "orderDate": "24/04/2026"
8    },
9    "signature": "{{computed_signature}}"
10 }
11

```

Sample Success Response

```

1  {
2    "responseCode": "000",
3    "responseDesc": "Operation completed successfully",
4    "transactionId": "TXN123456",
5    "storeId": "947842",
6    "orderId": "OnlineService1",
7    "reversalAmount": "25.00",
8    "merchantName": "Merchant Name",
9    "reversalId": "REV-001"
10 }
11

```

Sample Failure Response

```

1  {
2    "responseCode": "001",
3    "responseDesc": "System error occurred",
4    "transactionId": null,
5    "storeId": "947842",
6    "orderId": "OnlineService1",
7    "reversalAmount": null,
8    "merchantName": null,
9    "reversalId": ""
10 }

```

Response Parameters

Field	Description	Type
responseCode	Response code	String
responseDesc	Response description	String
transactionId	Original transaction ID	String
storeId	Merchant store ID	String
orderId	Merchant's order ID	String
reversalAmount	Final reversed amount	String
merchantName	Merchant display name	String
reversalId	Unique reversal transaction ID	String

6. Response Codes

6.1 Transaction Response Codes

Applicable to: **Initiate MA Transaction** and **Inquiry API**

Code	Description
000	Operation completed successfully
001	System error occurred
002	Configuration error
003	Database error
004	Network error
005	Invalid input format
006	Invalid amount
007	Amount limit exceeded
008	Invalid transaction type

009	Invalid store
010	Transaction not found
011	Transaction not paid
012	Duplicate request
013	Merchant validation failed
014	Insufficient balance
015	Transaction expired
016	Unauthorized
017	Forbidden
018	Authentication failed
019	Invalid mobile number
020	No transaction handlers enabled for execution
021	Merchant not found
022	Account is suspended
023	Account status is invalid
024	FRI not found
025	Account balance unavailable
026	Account limit exceeded
027	Transaction blocked by prevention service
028	Prevention checks failed
029	Failed to calculate transaction deductions
030	Failed to create transaction
031	Payment debit operation failed

032	Payment debit was not successful
033	Transaction data is missing
034	Merchant data is missing
035	Transaction request is missing
036	Transaction deductions are missing
037	Account information is missing
038	Account prechecks failed
039	Payment confirmation failed
040	Payment confirmation pending (Async)

6.2 Reversal-Specific Response Codes

In addition to the standard codes above (000–018), the Reversal API may also return:

Code	Description
101	Transaction is already reversed
102	Reversal is in process
103	Reversal is scheduled
104	Reversal amount exceeds transaction amount
105	Sum of reversal amounts exceeds transaction amount
106	Invalid NIC
107	Transaction already reversed or in progress
108	Transaction type not supported for reversal

109	Invalid or multiple reversal configurations found
110	Reversal amount exceeds maximum limit configured
111	Error in processing of the reversal request
112	Transaction paid but not settled, cannot reverse

6.3 Auth Response Codes

Applicable to OPS signature validation layer:

HTTP Status / Code	Description
200 / 00	JWT and signature are both valid
401 / 401	Authentication error — missing or invalid JWT
200 / 07	JWT valid but signature failed — payload tampered or wrong <code>apiSecret</code>